Monedero — Guion de exposición

Monedero electrónico prepago de circuito cerrado. Gestión de la Calidad del Software, UFG.

Duración: 9 minutos más preguntas. Expositores: Steven (0:00 a 3:00), Bayron (3:00 a 6:00) y David (6:00 a 9:00).

Preparación (antes de empezar)

Abrir Docker Desktop y esperar "Engine running". En C:\src\monedero ejecutar: dart run tool/up.dart

En otra consola: cd apps/app y luego flutter run -d web-server --release --web-port 5000 --web-hostname localhost

Dejar abiertas las pestañas localhost:5000 (app), localhost:8025 (Mailpit) y el diagrama de docs/architecture.md. Tener lista otra consola con: dart run tool/demo.dart --pausa

Steven — Introducción y arquitectura (0:00 a 3:00)

"Nuestro proyecto es Monedero: una tarjeta prepago de circuito cerrado, como la de un supermercado. Se recarga en caja y se gasta solo dentro del comercio."

"Aunque es académico, lo diseñamos como si manejara dinero real: no se puede perder ni duplicar un centavo."

"Hay tres usuarios: el titular, que ve su saldo y su tarjeta; la caja, que cobra y recarga; y la administración, que emite tarjetas y revisa que todo cuadre."

"La arquitectura tiene tres piezas: una app Flutter para Android y web, una API en Dart que aplica las reglas, y PostgreSQL, que es la única fuente de verdad del saldo. En local todo corre con Docker y cuesta cero."

"Para producción ya está diseñado: un servidor en Contabo, HTTPS con Caddy, respaldos continuos y, más adelante, tarjetas físicas por NFC. Hoy las tarjetas se simulan con un código QR."

Mostrar: el diagrama de arquitectura (App, API, PostgreSQL).

Bayron — Reglas del dinero y vistas de la app (3:00 a 6:00)

"Protegemos el dinero en tres capas: la app valida para ayudar al usuario, la API decide y la base de datos es la última barrera."

"El dinero se guarda en centavos enteros, nunca con decimales, para evitar errores de redondeo."

"Las recargas son solo múltiplos de 5 dólares, no se permite sobregiro y los cobros mayores de 25 dólares piden el PIN del titular."

"Cada operación es atómica y lleva una clave de idempotencia: si la red falla y la caja reintenta, no se cobra dos veces."

"Los movimientos no se pueden modificar ni borrar, y el saldo siempre debe ser igual a la suma de los movimientos. A eso le llamamos conciliación."

"Cada tarjeta tiene una firma criptográfica, así que nadie puede fabricar una tarjeta válida."

Mostrar en localhost:5000, aclarando que son vistas de diseño con datos de ejemplo y que la lógica real ya funciona en la API: el inicio con "API conectada"; la vista del titular (tarjeta, saldo, QR, movimientos, reportar pérdida); la caja (tres formas de leer la tarjeta, cobrar, recargar, comprobante y errores); y la administración (conciliación en $0.00, tarjetas y terminales).

David — Demostración en vivo y calidad (6:00 a 9:00)

"Ahora vamos a usar el sistema de verdad. Este programa hace exactamente las mismas llamadas que hará la app, contra la API y la base de datos reales."

Ejecutar dart run tool/demo.dart --pausa y presionar Enter en cada paso: registrar un titular (el código llega a Mailpit); emitir la tarjeta; recargar $20; cobrar $7.35 (saldo $12.65); intentar un sobregiro de $15 (rechazado); la misma recarga enviada 5 veces se aplica una sola vez; una tarjeta falsificada se rechaza; reportar la pérdida (queda bloqueada); reponer la tarjeta con el mismo saldo; y ver la conciliación con diferencia total de $0.00.

"Tenemos pruebas automáticas contra PostgreSQL real: 1,000 cobros al mismo tiempo sin saldo negativo, la misma operación cinco veces aplicada una sola vez, una reposición que falla a la mitad y se revierte completa, la base rechazando modificar un movimiento y la firma falsa rechazada."

"Trabajamos con calidad de proceso: Git y GitHub con ramas, Conventional Commits, integración continua, registro de decisiones técnicas, bitácora de sesiones y una definición de terminado."

"Lo siguiente es conectar las vistas a la API, la app en Android, las tarjetas NFC reales y el despliegue. Gracias, ¿preguntas?"

Preguntas probables

¿Por qué no Firebase? Necesitamos transacciones con bloqueo, restricciones en la base y conciliación exacta; PostgreSQL lo hace de forma nativa.

¿Qué pasa si se cae el internet? No se cobra, porque sin conexión no hay forma segura de evitar el doble gasto. La app solo muestra el último saldo marcado como "no actualizado".

¿Y si roban la tarjeta? El titular la reporta con un código por correo y queda bloqueada de inmediato; los cobros mayores de $25 piden PIN y la reposición conserva el saldo.

¿Las pantallas ya funcionan? Hoy son vistas de diseño; la lógica completa ya funciona en la API, como se vio en la demostración.

¿Cuánto cuesta? La demo cuesta $0. En producción, un VPS barato y almacenamiento para respaldos.

Si algo falla

Si la demo no inicia sesión: dart run tool/seed.dart. Si la API no responde: dart run tool/up.dart. Si la app web no carga: mostrar las capturas de docs/img.